DOSSIER DE GOUVERNANCE

Continuité, risques et conformité

BIA • PCA/PRA • EBIOS RM ateliers 2 à 5 • NIS2/DORA • Recommandations

0 L'essentiel en une page

Ce dossier couvre la partie **gouvernance** exigée par le sujet (section 3.5) : ce que coûte un arrêt (BIA), comment on reprend (PCA/PRA), l'analyse de risques EBIOS RM (ateliers 2 à 5 et mise à jour de l'atelier 1), la conformité NIS2/DORA et nos recommandations à la direction.

Tous les chiffres ont été mesurés le 21/10/2026 sur la dernière version du code (main, 17 services Docker), en provoquant de vraies pannes et de vraies attaques. Ils sont à refaire sur le PC Hôte avant la soutenance.

Ce qu'on a mesuré	Résultat	Ce que ça veut dire
Crash de l'API de paiement	**0,2 s d'indisponibilité**	Docker relance le conteneur tout seul
Crash de la base PostgreSQL	**0,3 s, 0 paiement accepté perdu**	PostgreSQL rejoue son journal (WAL) ; 2 paiements refusés proprement
Procédure Ansible restore_database	**9,6 s**, contrôles OK	Contrôle d'intégrité, relance de l'API, paiement de preuve
Flood de Clara (400 requêtes, 690/s)	**90,2 % rejetées**, service maintenu	La limitation de débit absorbe l'attaque, même avec de fausses IP…
Client légitime extérieur pendant le flood	**4 paiements sur 25 refusés**	… mais touche aussi les vrais clients (même IP vue par le WAF)
Force brute sur un compte (40 essais)	**verrouillé après 5 échecs**	35 tentatives sur 40 refusées, événement LOGIN_LOCKED journalisé
Boucle détection → réparation (flood)	**≈ 35 s, sans humain**	Attaque → détection ML → playbook Ansible → paiement de preuve (mesuré par l'équipe le 22/10)

À retenir : 1 minute d'arrêt coûte environ **346 € de commissions** et bloque **28 800 € de paiements** pour nos marchands. Nos reprises mesurées se comptent en secondes. Depuis le 20/10, l'équipe a mis en service la **réparation automatique pilotée par la détection**, l'authentification, et corrigé l'usurpation d'IP. La boucle répare une attaque en **environ 35 s**, sous l'objectif de 2 min ; notre point faible structurel reste l'absence de **sauvegardes**.

1 Analyse d'impact (BIA) : combien coûte un arrêt des paiements

Les chiffres de l'entreprise sont **fictifs** (le sujet l'impose) mais réalistes pour un prestataire de paiement de taille moyenne. Ils servent à fixer nos objectifs de reprise et à relier la gravité EBIOS à un coût.

1.1 Hypothèses

Hypothèse	Valeur	Commentaire
Volume moyen en journée	600 paiements / min	Pic : 1500 / min (soldes, fin de mois)
Montant moyen d'un paiement	48 €	Panier e-commerce moyen
Commission perçue	1.2 %	Notre chiffre d'affaires sur chaque paiement
Pénalité partenaires (SLA 99,9 %)	5 000 € / heure entamée	Au-delà de 15 min d'arrêt
Cellule de crise	4 personnes x 90 €/h	Mobilisée au-delà de 15 min

1.2 Coût selon la durée de l'arrêt

Durée	Paiements bloqués	Flux bloqué (marchands)	Commissions perdues	Pénalités	Crise	Coût pour nous	Gravité EBIOS
0,3 s (crash mesuré)	3	144 €	2 €	—	—	**2 €**	G1
35 s (boucle mesurée)	350	16 800 €	202 €	—	—	**202 €**	G2
2 min (RTO cible)	1 200	57 600 €	691 €	—	—	**691 €**	G2
15 min	9 000	432 000 €	5 184 €	—	90 €	**5 274 €**	G3
1 h	36 000	1 728 000 €	20 736 €	5 000 €	360 €	**26 096 €**	G3
4 h	144 000	6 912 000 €	82 944 €	20 000 €	1 440 €	**104 384 €**	G4

- **1 minute d'arrêt** = 346 € de commissions perdues et 28 800 € de paiements bloqués (au pic : 864 € et 72 000 €).
- **Au-delà de 15 min**, les pénalités partenaires et la cellule de crise s'ajoutent : le coût change d'échelle.
- **Le vrai risque n'est pas que financier** : un arrêt long est un incident majeur au sens de DORA (notification à l'autorité), et les marchands peuvent partir chez un concurrent. Ces impacts ne sont pas chiffrés ici.

1.3 Objectifs de reprise retenus

Objectif	Cible	Justification	Mesuré aujourd'hui
RTO : durée maximale d'arrêt	**2 min**	Coût encore limité (~700 €), avant les pénalités	0,2 s (crash API), 0,3 s (crash base)
RPO : perte de données maximale	**0 paiement validé**	Un paiement confirmé perdu = litige client	0 paiement accepté perdu sur crash de la base
Temps de détection	< 30 s	Laisser le temps de réparer et vérifier dans le RTO	Au plus tôt 10 s par conception (2 analyses de 5 s), compris dans les 35 s mesurés
RTO d'une attaque (détection → réparation → preuve)	**2 min**	C'est ce que verra le jury	**≈ 35 s** (flood, mesuré le 22/10)

2 Plan de continuité et de reprise (PCA/PRA)

Le **PCA** décrit comment le service **continue** pendant l'incident, le **PRA** comment on le **rétablit** ensuite. Chaque procédure s'appuie sur un composant qui existe dans le dépôt.

2.1 Scénarios couverts

Scénario	Ce qui se passe	Réponse (automatique ou procédure)	Résultat mesuré	Couvert ?
S1 - Flood sur /api/pay (SC-01, ER1)	Attaque depuis le PC Attaquant (red_attack.py --mode flood)	Double WAF : limitation de débit du WAF applicatif (40 requêtes / 2 s par client), puis automatiquement : détection ML → alerte dos → limitation dans le WAF nginx	90,2 % de l'attaque rejetée, service maintenu. Boucle complète : ≈ 35 s. 4/25 clients légitimes refusés	**Partiel**
S2 - Crash de l'API	Le processus de payment-api s'arrête	Docker relance le conteneur (restart: unless-stopped)	0,2 s d'indisponibilité	**Fait**
S3 - Crash de la base	Arrêt brutal de PostgreSQL	Docker relance PostgreSQL, rejeu du journal (WAL)	0,3 s, 2 paiements refusés, 0 paiement accepté perdu	**Fait**
S3b - Force brute sur un compte (ER3)	Des milliers de mots de passe testés sur /auth/login	Verrouillage après 5 échecs par compte, journalisé (LOGIN_LOCKED)	35 tentatives sur 40 refusées (429)	**Fait**
S4 - Doute sur l'intégrité des données	Après un incident, il faut prouver que les comptes sont justes	Playbook *restore_database*	9,6 s, contrôles OK, paiement de preuve accepté	**Fait**
S5 - Intrusion (outil d'attaque reconnaissable, SC-02)	sqlmap, nikto… ou accès refusés (403)	Automatique : alerte intrusion → *block_source* (User-Agent) puis *restore_database*	Règle appliquée, mais le WAF redémarre (quelques secondes de coupure). Boucle à chronométrer	**Partiel**
S6 - Service arrêté que Docker ne relance pas	Un service de la liste blanche n'est plus running ou healthy	Automatique : alerte crash → *restart_service* (seulement s'il est encore KO)	Mécanisme en place dans le code	**À mesurer**
S7 - Perte du PC Hôte ou du disque	Matériel en panne, volume de données perdu	Aucune : pas de sauvegarde, pas de second PC	Non couvert	**Absent**

2.2 Procédures de reprise (playbooks Ansible)

Les playbooks tournent dans le conteneur *remediation*. Ils ne peuvent agir sur Docker qu'à travers un proxy qui n'autorise **que la lecture et le redémarrage**, et seulement sur une liste blanche de services (jamais la base, la supervision ou les proxys).

Playbook	Déclencheur	Étapes	Preuve de reprise	Durée
restart_service	Service dégradé (S2, S6)	1. Vérifie que le service est autorisé 2. Le redémarre 3. Attend qu'il soit sain (60 s max)	Paiement de test via le WAF	quelques s
block_source	Source malveillante (S5)	1. Contrôle la valeur (refuse les IP Docker) 2. Écrit la règle (IP, User-Agent ou limitation) 3. Recharge le WAF	Requête de contrôle bloquée (403)	quelques s
restore_database	Doute sur les données (S4)	1. Contrôles d'intégrité 2. Redémarre l'API 3. Paiement de test 4. Nouveaux contrôles	Contrôles OK avant et après + paiement accepté	9,6 s

Attention : Depuis le 20/10 au soir, ces playbooks sont lancés **automatiquement par le moteur de remédiation** à partir des alertes du détecteur ML (jeton obligatoire, 90 s entre deux actions sur un même service, 3 actions au maximum en 10 min). Les commandes make restart, make block et make restore restent pour les tests : **ne jamais les utiliser pendant la démo** (intervention humaine = 0/5 sur l'axe 1).

2.3 Qui fait quoi (RACI)

R = réalise, **A** = rend des comptes (un seul par ligne), **C** = consulté, **I** = informé.

Activité	Moteur auto	Antoine (Dev)	Mélanie (Infra)	Clara (Cyber)	Agathe (IA)	Timéo (Gouv.)
Détecter l'anomalie	R	I	C	C	A	I
Décider de la réparation	R	A	C	C	C	I
Redémarrer / restaurer (playbooks)	R	C	A	I	—	I
Bloquer une source dans le WAF	R	I	C	A	—	I
Vérifier par une transaction test	R	A	C	—	—	I
Rotation des secrets (fuite)	—	C	C	A/R	—	I
Communication, notification DORA	—	C	C	C	C	A/R

Activité	Moteur auto	Antoine (Dev)	Mélanie (Infra)	Clara (Cyber)	Agathe (IA)	Timéo (Gouv.)
Retour d'expérience (post-mortem)	—	R	R	R	R	A

2.4 Risques résiduels après reprise

Risque résiduel	Pourquoi	Décision
Un seul PC Hôte	Tout tourne sur une machine : panne matérielle = arrêt total (S7)	Accepté pour le prototype, à traiter (R2)
Pas de sauvegarde de la base	restore_database contrôle et relance, mais ne restaure rien. Volume perdu = données perdues	Accepté pour le prototype, à traiter (R2)
Blocage par IP inutilisable	Sous Docker Desktop, le WAF voit tous les clients extérieurs avec la même IP (10.77.10.1). Le playbook refuse donc de bloquer ces adresses	Bloquer par User-Agent ou par limitation de débit (R3)
Effet collatéral de la limitation de débit	Pendant un flood, des clients légitimes extérieurs sont refusés (4/25 mesurés)	Accepté temporairement, à traiter (R3)
Verrouillage de compte détournable	Tous les clients extérieurs ont la même IP vue par le WAF : 5 faux mots de passe suffisent à bloquer le compte d'un vrai client	Accepté pour le prototype (R3)
Paiements anonymes acceptés	REQUIRE_AUTH=false : un paiement sans jeton passe, sans débit de compte (utile au générateur de trafic)	À dire au jury ; à activer en production
Boucle mesurée une seule fois	≈ 35 s sur le flood ; l'intrusion et le service arrêté n'ont pas été chronométrés	Refaire la mesure le jour J (R1)
Limitation de la remédiation commune à tous	La règle posée après un flood limite tous les clients extérieurs (même IP vue par le WAF) et n'est jamais retirée automatiquement	make unblock après l'incident ; retrait automatique à prévoir
Réponse choisie par des règles	Le modèle juge l'anomalie, mais une alerte ne part que si une cause est reconnue (conteneur arrêté, refus 429/503, outil d'attaque)	Accepté : à expliquer au jury
Blocage = redémarrage du WAF	Appliquer une règle coupe le service quelques secondes pour tout le monde	À corriger (rechargement sans redémarrage)

3 EBIOS RM : ateliers 2 à 5

Version courte, reliée au démonstrateur. Elle reprend les valeurs métier (VM1 à VM4), les sources de risque (SR1 à SR3) et les scénarios (SC-01 à SC-04) du rapport de Clara, pour que les deux documents parlent la même langue.

3.1 Atelier 2 : qui pourrait attaquer, et pourquoi

Source de risque	Ce qu'elle veut (objectif visé)	Motivation	Ressources	Retenue ?
SR1 - Cybercriminel	Rendre les paiements indisponibles pour exiger une rançon	Forte (argent)	Moyennes (outils de flood)	Oui : scénario de la démo
SR2 - Fraudeur	Passer des paiements frauduleux, rejouer des transactions	Forte (argent)	Faibles à moyennes	Oui
SR3 - Initié ou dépendance piégée	Voler les secrets ou les données clients	Moyenne	Élevées (accès interne, chaîne logicielle)	Oui
Hacktiviste	Nuire à l'image de l'entreprise	Faible pour un acteur du paiement	Variables	Non (pertinence faible)

3.2 Atelier 3 : par où passerait l'attaquant (scénarios stratégiques)

Partie prenante	Pourquoi elle compte	Niveau de confiance
Clients (navigateurs)	Seul accès autorisé à la plateforme, via le WAF	Faible : n'importe qui
Marchands partenaires	Reçoivent les paiements, subissent l'arrêt	Moyen
Équipe d'exploitation	Détient la clé SOPS et l'accès au PC Hôte	Élevé mais risque d'erreur ou de malveillance
Fournisseurs logiciels	Images Docker, paquets Python et npm, modèle d'IA téléchargé au démarrage	Moyen : versions figées pour les images, pas pour tout

- **Scénario stratégique 1** (SR1 → ER1) : le cybercriminel sature l'API de paiement depuis Internet ; les marchands ne sont plus payés.
- **Scénario stratégique 2** (SR2 → ER2, ER3) : le fraudeur passe des paiements sans être authentifié ou rejoue une transaction.
- **Scénario stratégique 3** (SR3 → ER4, ER6) : une dépendance piégée ou un initié vole les secrets, puis détourne le moteur de réparation.

3.3 Atelier 4 : scénario opérationnel SC-01, DDoS applicatif L7 (attaque de Clara)

Étape	Ce que fait l'attaquant	Ce que fait la plateforme (constaté)
1. Repérage	Trouve la page publique et la route /api/pay	Le WAF ne laisse passer que 3 routes de l'API (le reste répond 403)
2. Attaque	Lance red_attack.py --mode flood : 400 requêtes, 25 en parallèle, 690 requêtes/s	Le WAF applicatif répond 429 à 90,2 % des requêtes
3. Effet	Cherche à saturer l'API et la base	Le service reste disponible ; 4 clients légitimes sur 25 refusés au passage
4. Tentative de contournement	Change de fausse IP à chaque requête (en-tête X-Forwarded-For)	Sans effet : le WAF réécrit l'en-tête (20 requêtes sur 60 bloquées malgré 60 fausses IP)
5. Réparation automatique	—	Détection ML (2 analyses de 5 s) → alerte dos → playbook *block_source* (limitation dans le WAF) → paiement de test : **≈ 35 s, sans humain**

Vraisemblance : V3 (très vraisemblable) : l'outil est simple. **Gravité résiduelle : G2** : l'attaque est absorbée, mais des clients légitimes sont gênés. Les autres scénarios du rapport de Clara sont traités ainsi : SC-02 (fuzzing) par la validation des données, SC-03 (fuite d'informations) par le masquage des erreurs, SC-04 (chaos) uniquement en test interne.

3.4 Atelier 5 : plan de traitement

Risque	Mesure	Responsable	Échéance	Risque résiduel
ER1 Flood	**Fait** : X-Forwarded-For réécrit par le WAF ; boucle détection → playbooks mesurée à ≈ 35 s. **Reste** : retrait automatique de la limitation	Mélanie, Clara	Après le workshop	Effet collatéral sur les clients extérieurs (Docker Desktop)
ER2 Paiement faux	**Fait** : débit du solde et refus si solde insuffisant. **Reste** : clé d'idempotence obligatoire	Antoine	Après le workshop	Paiements anonymes sans débit
ER3 Compte client	**Fait** : service auth (scrypt, JWT) et verrouillage après 5 échecs. **Reste** : REQUIRE_AUTH=true en production	Antoine	Après le workshop	Verrouillage détournable contre un vrai client
ER4 Secrets	Passer des variables d'environnement aux fichiers /run/secrets ; ne plus partager le secret JWT avec payment-api (RS256)	Clara, Antoine	Si le temps le permet	Secrets visibles avec docker inspect
ER6 Moteur	**Fait** : routes /waf protégées par jeton ; moteur construit (alertes avec jeton, liste blanche, garde-fous)	Mélanie	Fait	Faible
ER7 PC Hôte	Sauvegarde quotidienne de la base hors du PC + test de restauration	Mélanie	Après le workshop	Accepté pour le prototype

4 Atelier 1 mis à jour : ce qui nous protège vraiment

La version du 19/10 annonçait des protections prévues dans l'architecture. Voici ce qui existe **réellement** dans le code au 21/10, vérifié par des tests sur la stack. Cette version remplace la colonne « Ce qui nous protège déjà ».

Risque	Gravité	Protection annoncée le 19/10	État réel au 21/10 (testé)	Statut
ER1 Les paiements ne passent plus	G3	WAF qui limite le débit, détection ML, IP bannie	Limitation de débit non contournable (90,2 % du flood rejeté, même avec de fausses IP) ; boucle ML → limitation dans le WAF en ≈ 35 s ; bannir une IP impossible sous Docker Desktop	**Partiel**
ER2 Un paiement est faux	G4	Paiement en une requête avec contrôle du solde	Paiement authentifié : solde débité, refus si insuffisant (testé). Contrainte « solde positif » et clé d'idempotence unique en base. Mais clé facultative et paiements anonymes acceptés	**Partiel**
ER3 Un pirate entre dans un compte	G3	Essais limités sur /login, détection ML	Service auth en place : mots de passe scrypt, jeton JWT (falsifié = 401), verrouillage après 5 échecs (35/40 essais refusés, testé)	**Fait**
ER4 Vol de nos secrets	G4	Secrets chiffrés SOPS, clé JWT seulement dans auth	SOPS + age en place, .env hors de Git. Secrets en variables d'environnement ; secret JWT partagé entre auth et payment-api	**Partiel**
ER5 Fuite des données clients	G3	Règles anti-injection du WAF, base inaccessible	WAF : liste blanche (docs, metrics, routes d'administration → 403, testé) ; base non publiée ; erreurs masquées (CWE-209). /api/waf/status reste public	**Fait**
ER6 Moteur de réparation détourné	G4	Accès Docker limité, aucune commande de l'extérieur	Moteur en place : alertes avec jeton, liste blanche, 90 s entre deux actions, 3 au maximum en 10 min ; proxy limité au redémarrage ; routes /waf protégées (sans jeton = 401, testé)	**Fait**

Risque	Gravité	Protection annoncée le 19/10	État réel au 21/10 (testé)	Statut
ER7 Le PC Hôte tombe	G3	Playbook pour tout réinstaller	Relance automatique des conteneurs (0,2 s et 0,3 s mesurés). Pas de sauvegarde, pas de second PC	**Partiel**

5 Conformité NIS2 / DORA

Pour une entité financière, **DORA** (règlement (UE) 2022/2554) s'applique en priorité : c'est la règle spécifique au secteur, NIS2 (directive (UE) 2022/2555) reste la référence générale. Ce tableau fusionne la section DORA du rapport de Clara avec l'analyse de gouvernance.

Exigence DORA	Ce que le démonstrateur prouve	Ce qu'il ne peut pas prouver	Statut
Cadre de gestion du risque TIC (art. 5 à 9)	Inventaire des actifs (architecture, plan d'adressage), analyse EBIOS RM, durcissement des conteneurs, réseaux isolés	Implication de l'organe de direction, politique formelle validée	**Partiel**
Détection (art. 10)	Isolation Forest appliqué toutes les 5 s aux journaux de la plateforme ; supervision Prometheus, Loki, Grafana	Performance en conditions réelles (faux positifs sur la durée)	**Fait**
Réponse et rétablissement (art. 11)	Réponse automatique en ≈ 35 s (flood) avec preuve par transaction ; reprises sur crash en 0,2 s et 0,3 s	Fiabilité sur la durée (une seule mesure de bout en bout)	**Fait**
Sauvegarde et restauration (art. 12)	Aucune perte sur crash de la base (journal WAL)	Sauvegardes, restauration depuis une copie, site de secours	**Absent**
Gestion et classification des incidents (art. 17 et 18)	Journal d'audit en base (service_audit_logs, dont LOGIN_LOCKED), journaux centralisés dans Loki	Processus de classification, critères d'incident majeur	**Partiel**
Notification des incidents majeurs (art. 19)	Les données nécessaires existent (heure, durée, impact mesuré)	Notification réelle à l'autorité (en France : l'ACPR)	**Absent**
Tests de résilience (art. 24 à 27)	Tests de panne réels (crash API et base), attaques simulées, make chaos	Tests de pénétration avancés (TLPT) par un tiers	**Partiel**
Risque lié aux prestataires TIC (art. 28 et suivants)	Versions d'images Docker figées	Registre des prestataires, clauses contractuelles, stratégie de sortie	**Absent**
Partage d'informations (art. 45)	—	Participation à des échanges sur les menaces	**Absent**

Côté NIS2, les mesures de l'article 21 recoupent ce tableau ; la notification de l'article 23 (24 h, 72 h, un mois) n'est pas démontrable avec un prototype.

Attention : Corrections à reporter dans le rapport de Clara : le blocage **redémarre** le WAF ; la clé d'idempotence est **facultative** ; le RTO mesuré (0,2 s le 21/10) vient du **redémarrage automatique de Docker**, pas de la réparation pilotée par le ML.

6 Recommandations à la direction

Ce que nous ferions dans un contexte industriel, par ordre de priorité. Chaque recommandation répond à un risque mesuré. **Arbitrage proposé :** R1 et R4 d'abord (peu coûteux, fort impact sur la conformité DORA), puis R3, puis R2 qui demande un budget d'infrastructure.

#	Recommandation	Pourquoi (preuve)	Effort	Indicateur de succès
R1	**Éprouver la réparation automatique** : la chronométrer, la tester chaque mois et retirer automatiquement les limitations posées	La boucle répare un flood en ≈ 35 s, mais n'a été mesurée qu'une fois ; la limitation posée reste en place pour tous les clients	Faible	Incident détecté et réparé sans humain en moins de 2 min, mesuré en test mensuel
R2	**Sauvegarder la base hors du serveur et ajouter un second site**	Une panne du PC ou du disque = perte totale (S7). DORA art. 12 non couvert	Moyen à élevé (budget infra)	RPO de 15 min garanti, restauration testée chaque trimestre
R3	**Placer le pare-feu en frontal et exiger l'authentification** (REQUIRE_AUTH, MFA)	Le WAF ne voit pas les vraies IP : la limitation touche les vrais clients (4/25) et un attaquant peut verrouiller le compte d'un client	Moyen	0 client légitime refusé pendant un flood
R4	**Formaliser la gestion des incidents DORA** : classification, notification, exercices de crise	Les données existent mais aucun processus n'existe pour notifier dans les délais	Faible	Exercice de crise semestriel, notification simulée en moins de 4 h
R5	**Maîtriser la chaîne logicielle** : inventaire des composants (SBOM), analyse des images, registre des prestataires	Le service IA télécharge un modèle sur Internet à chaque démarrage ; pas de registre des fournisseurs	Faible à moyen	100 % des images analysées avant déploiement